`int` considered harmful

A case against the default integer, and a type that fixes it

Peter Neiss

26 July 2026
revised 6 October 2026

Abstract

C++'s `int` is not a number. It is a platform-dependent bit pattern whose behaviour is undefined at both ends of its range, which silently changes width and signedness in the middle of expressions, which rounds without being asked, and which carries no record of what it is counting. Every one of those properties is a defect, and all of them are load-bearing in code that ships today.

This paper does two things. First it puts `int` in the dock: eleven hazards, each a complete program, compiled and run, with the real output reproduced. One of them causes GCC to emit *no machine code at all* for `main`. It then makes the same case, more briefly, against reaching for `double` instead — the reflex fix that trades one kind of silent wrongness for another.

Second, it presents `inside`, a header-only C++23 library in which the range and the step size of a number live in its type. Arithmetic widens its result type at compile time, so overflow is not merely detected but unreachable; division by zero is a value you can test rather than a signal that kills the process; fractional values are exact rationals rather than binary approximations; and what happens at the edge of a range is a policy you state in the declaration. On performance we find no single answer: against code providing the same guarantee, `inside` is by turns *faster* (it discharges at compile time checks that hand-written code must perform at runtime), exactly free (byte-identical machine code), and substantially slower (exact rational arithmetic, and transcendentals correctly rounded at the full resolution of a `double`). We report all three, with the programs that produced them.

A note on evidence. Every program in this paper lives in `docs/int-considered-harmful/` in the `inside` repository. Every output block was captured by running the program, not written by hand. All measurements come from the repository's generated `docs/performance.md` or from programs in that directory; none are estimates. Compiler is GCC 15.2.0 on x86-64 Linux unless stated otherwise. `inside` is under development and not yet ready for production use; its API may change. This revision (October 2026) updates every `inside` program, output and measurement to the current library; the `int` and `double` exhibits reproduce unchanged.

On AI assistance. This paper was drafted with Claude Code, which wrote the example programs, compiled and ran them to capture the outputs reproduced here, extracted the benchmark figures from the repository's generated reports, and produced the typeset document. The `inside` library itself was likewise developed with Claude Code. The argument, the framing and the final review are the author's; the claims were checked against the compiler and the repository rather than taken on the model's word, and the reader is invited to do the same — every program is included so that they can.

Contents

1 The thesis **4**

2 The indictment **4**
2.1 Signed overflow is undefined, and the optimiser believes it . . . 4
2.2 The same assumption deletes an entire function . . . 5
2.3 Two's complement is asymmetric . . . 6
2.4 Division has two undefined inputs and no error value . . . 6
2.5 Mixed signed/unsigned comparison . . . 6
2.6 Small types do not stay small . . . 7
2.7 Division truncates, and not in the direction you expect . . . 7
2.8 Shifts . . . 8
2.9 Narrowing is silent . . . 8
2.10 The midpoint bug . . . 9
2.11 The semantic hazard: `int` does not know what it counts . . . 9

3 "Just use `double`" **10**
3.1 Decimal fractions are not representable . . . 10
3.2 The error accumulates . . . 10
3.3 Addition is not associative . . . 11
3.4 Integers stop counting at 2^{53} . . . 11
3.5 `NaN` breaks equality and ordering . . . 11

4 Why the usual defences fall short **12**

5 `inside`: putting the range in the type **13**
5.1 Arithmetic widens, so overflow is unreachable . . . 13
5.2 Division by zero is a value, not a signal . . . 14
5.3 Rounding is stated, not inherited . . . 14
5.4 The edge is a policy you declare . . . 15
5.5 Negation reflects the type . . . 16
5.6 The scale is part of the type . . . 16
5.7 Bounded indices . . . 17
5.8 No 64-bit ceiling . . . 18

6 A worked migration **18**

7 Correctness **20**
7.1 Errors caught before the program runs . . . 20
7.2 The error vocabulary . . . 21
7.3 Exactness . . . 21
7.4 Reproducibility . . . 22

8 Performance **22**
8.1 The fast path is byte-identical to native . . . 23
8.2 What it costs, and against what . . . 24
8.3 The like-for-like comparison . . . 24
8.4 Where are the checks? . . . 25
8.5 The generated tables . . . 26
8.6 What correct rounding costs . . . 27
8.7 Reading these numbers honestly . . . 28

9 When not to use this **29**

10 Conclusion **29**

A Building the examples **30**

B Program index **31**

C References **31**

1 The thesis

Dijkstra's 1968 letter did not argue that `goto` produced wrong answers. It argued that `goto` destroyed the correspondence between the program on the page and the process it described, so that a reader could no longer reason about what was true at a given point. The objection was about *tractability*, not arithmetic.

The case against `int` is narrower and, in a sense, worse. `int` does produce wrong answers. It produces them silently, it produces them in code that compiles without a warning at maximum diagnostic settings, and — the part practitioners consistently underestimate — it produces them by causing the compiler to *delete code the programmer wrote*, including the code written to check for the very failure that occurs.

The root of it is a mismatch of models. The programmer thinks of `int` as $\mathbb{Z}$, the integers. The standard defines it as a finite set with undefined behaviour at the boundary, and permits the optimiser to assume the boundary is never reached. Those two models agree over the range where testing usually happens and diverge exactly where it usually does not.

What follows from the mismatch is a family of defects that no amount of discipline fully removes, because the information needed to check the operation — what range is this value allowed to occupy? what is it counting? what should happen if it leaves? — is not present in the program at the point where the operation happens. It exists only in the programmer's head and, if you are fortunate, in a comment.

`inside`'s proposal is to put that information in the type, where the compiler can act on it. The remainder of this paper establishes that the problem is real (§2–§3), that the usual defences do not solve it (§4), and that a type-level answer does (§5–§7), at a cost that is sometimes nothing at all and sometimes substantial (§8).

2 The indictment

2.1 Signed overflow is undefined, and the optimiser believes it

The first thing to establish is that signed overflow is not wraparound. It is undefined behaviour, which means the compiler may assume it does not occur and optimise on that assumption. Consider a function that asks whether adding one to a number makes it bigger.

```
// Does adding 1 to an int always make it bigger?
[[gnu::noinline]] static bool grows(int x)
{
  return x + 1 > x;
}
```

Since `x + 1 > x` can only be false if `x + 1` overflows, and overflow is undefined, the compiler concludes the expression is always true. It does not warn. It deletes the comparison:

```
$ g++ -std=c++23 -O0 h01_overflow_ub.cpp -o h01 && objdump -d h01
0000000000001149 <grows(int)>:
    1149:   endbr64
    114d:   push   %rbp
    114e:   mov    %rsp,%rbp
    1151:   mov    %edi,-0x4(%rbp)     <- argument stored...
    1154:   mov    $0x1,%eax           <- ...and ignored. Return 1.
    1159:   pop    %rbp
    115a:   ret
```

Note the optimisation level: `-O0`. This is not an aggressive release-build transformation that a debug build would protect you from. GCC folds the comparison during translation, before optimisation proper begins. The check cannot be written in a way that survives, because the

check is the thing being deleted.

```
INT_MAX            = 2147483647
grows(INT_MAX)     = yes, always
INT_MAX + 1        = -2147483648
```

The last two lines are worth reading together. The program reports that adding one to `INT_MAX` always produces a larger value, and then prints the smaller value it produced.

2.2 The same assumption deletes an entire function

That was a folded comparison. The assumption compounds. Here is a loop whose induction variable starts near the top of the range:

```
int main()
{
  int count = 0;

  for (int i = INT_MAX - 2; i > 0; ++i)
  {
    ++count;
    if (count > 10)
    {
      std::printf("still going after %d iterations -- loop never terminates\n", count);
      break;
    }
  }

  std::printf("iterations = %d\n", count);
```

At `-O0` the counter wraps to a negative value, `i > 0` goes false, and the loop exits after three iterations. At `-O2` the compiler reasons that `i > 0` can never become false without overflow, and overflow is undefined, therefore the loop is infinite, therefore the code after it is unreachable, therefore `main` has no reachable exit. What it emits is:

```
$ g++ -std=c++23 -O2 -Wall h02_overflow_loop.cpp -o h02
$ objdump -d h02

0000000000001040 <main>:
    1040:   endbr64
    1044:   cs nopw 0x0(%rax,%rax,1)
    104e:   xchg   %ax,%ax

0000000000001050 <_start>:
```

That is the whole of `main`: an endbranch landing pad and five bytes of alignment padding. There is no loop, no `printf`, and **no `ret` instruction**. Execution falls out of `main` into whatever follows it in the text segment:

```
$ ./h02
Segmentation fault (core dumped)          [exit 139]

$ ./h02_compiled_at_O0
iterations = 3
```

`-Wall` produced no diagnostic. The same source, the same compiler, two optimisation levels: one prints `3`, the other crashes before executing a single instruction of the program. This is the strongest available argument that undefined behaviour is not a theoretical concern about edge-case values. It is a compile-time contract, and breaking it does not corrupt your result — it

deletes your program.

2.3 Two's complement is asymmetric

There are 2^{31} negative `int` values and $2^{31} - 1$ positive ones. Negation therefore has no correct answer at one point in the range, and neither does absolute value.

```
  std::printf("INT_MAX            = %d\n", INT_MAX);
  std::printf("-INT_MIN           = %d   <- still negative\n", -m);
  std::printf("abs(INT_MIN)       = %d   <- still negative\n", std::abs(static_cast<int>(m)));

  // INT_MIN / -1 is also undefined: the true answer is INT_MAX+1.
  // On x86-64 it does not wrap, it raises SIGFPE. See h04_divzero.cpp.
```

```
INT_MIN            = -2147483648
INT_MAX            = 2147483647
-INT_MIN           = -2147483648   <- still negative
abs(INT_MIN)       = -2147483648   <- still negative
```

`std::abs` is a function whose entire purpose is to return a non-negative value, and it returns a negative one. It is not a bug in the standard library; there is no value it could return that would be correct.

2.4 Division has two undefined inputs and no error value

Floating point has `NaN` and infinities. Integer division has neither, so its two invalid inputs — division by zero, and `INT_MIN / -1`, whose true answer is `INT_MAX + 1` — cannot be represented as results. On x86-64 both raise `SIGFPE` and terminate the process.

```
$ ./h04_divzero 0
about to evaluate 10 / 0 ...
Floating point exception (core dumped)          [exit 136]

$ ./h04_divzero 1
about to evaluate -2147483648 / -1 ...
Floating point exception (core dumped)          [exit 136]
```

The second case is the one that surprises people: a division whose operands are both perfectly ordinary-looking, with no zero in sight, that kills the process. Any code that divides by a runtime value must check for two special cases, and almost none does.

2.5 Mixed signed/unsigned comparison

When an `int` meets an `unsigned` in a comparison, the usual arithmetic conversions make both unsigned. A negative value becomes an enormous positive one.

```
  // The classic: iterate backwards over an empty container.
  std::vector<int> v;   // empty
  std::printf("v.size()           = %zu\n", v.size());
  std::printf("v.size() - 1       = %zu   <- not -1\n", v.size() - 1);

  int visited = 0;
  for (std::size_t i = 0; i < v.size() - 1; ++i)
  {
    ++visited;
    if (visited > 3) break;   // escape hatch: this loop runs ~2^64 times
  }
  std::printf("loop body entered  = %d times on an EMPTY vector\n", visited);
```

```
-1 < 0u            = false
v.size()           = 0
v.size() - 1       = 18446744073709551615   <- not -1
loop body entered  = 4 times on an EMPTY vector
```

The loop iterates over an empty container. Without the escape hatch it would run roughly 1.8×10^{19} times, indexing far past the end of the allocation on the first pass. This is among the most common sources of exploitable memory-safety bugs in C++ code, and it arises from a *comparison*.

2.6 Small types do not stay small

Integral promotion means the operands of an expression are frequently not the types you wrote, and the result is frequently not the type of the operands.

```
  std::uint8_t a = 200;
  std::uint8_t b = 200;

  // Both operands are uint8_t; the product is not.
  std::printf("uint8 * uint8 -> int? %s\n",
              std::is_same_v<decltype(a * b), int> ? "yes" : "no");
  std::printf("200 * 200          = %d\n", a * b);
  std::printf("stored back in u8  = %u   <- truncated\n",
              static_cast<std::uint8_t>(a * b));

  // char signedness is not specified by the standard.
  std::printf("char is signed?    %s (platform-dependent)\n",
              std::is_signed_v<char> ? "yes" : "no");

  // Widths are minimums, not guarantees.
  std::printf("sizeof(int)        = %zu bytes here; the standard promises >= 2\n",
              sizeof(int));
```

```
uint8 * uint8 -> int? yes
200 * 200          = 40000
stored back in u8  = 64   <- truncated
char is signed?    yes (platform-dependent)
sizeof(int)        = 4 bytes here; the standard promises >= 2
```

Three separate portability hazards sit in that output. The multiplication of two 8-bit values happens in `int`, so the intermediate is fine but the store truncates. The signedness of bare `char` is implementation-defined — code that treats it as either is wrong somewhere. And `sizeof(int)` is 4 here but the standard guarantees only 2, which is not academic: it is 2 on a great deal of shipping embedded hardware.

2.7 Division truncates, and not in the direction you expect

```
  std::printf(" 7 / 2            = %d   (exact answer 3.5)\n", 7 / 2);
  std::printf("-7 / 2            = %d   (rounds toward zero, not down)\n", -7 / 2);
  std::printf(" 7 %% 2            = %d\n", 7 % 2);
  std::printf("-7 %% 2            = %d   <- negative remainder\n", -7 % 2);

  // Averaging three test scores: 90, 90, 91 -> 90, not 90.33
  int total = 90 + 90 + 91;
  std::printf("mean(90,90,91)     = %d   (true mean 90.333...)\n", total / 3);
```

```
 7 / 2             = 3   (exact answer 3.5)
-7 / 2             = -3   (rounds toward zero, not down)
 7 % 2             = 1
-7 % 2             = -1   <- negative remainder
mean(90,90,91)     = 90   (true mean 90.333...)
```

Integer division rounds toward zero, which is neither floor nor round-to-nearest. It is the only rounding mode available, it is applied silently, and it is the wrong one for most arithmetic people actually want: computing an average, splitting a bill, scaling a value by a percentage. The remainder is negative for negative operands, which breaks the common idiom `if (x % n == k)` for indexing into cyclic structures.

2.8 Shifts

```
  volatile int one = 1;
  volatile int shift = 31;

  // Shifting a 1 into the sign bit of a 32-bit int is UB.
  std::printf("1 << 31            = %d   <- undefined behaviour\n", one << shift);

  // Shifting by >= the width is UB, not "zero".
  // (Left as a comment so this program stays runnable.)
  //   one << 32;   // undefined
  //   one << -1;   // undefined

  // Right-shifting a negative value was implementation-defined before C++20.
  std::printf("-8 >> 1            = %d\n", -8 >> 1);
```

```
1 << 31            = -2147483648   <- undefined behaviour
-8 >> 1            = -4
```

Shifting a bit into the sign position of a signed type is undefined; so is shifting by an amount greater than or equal to the width, and shifting by a negative amount. The value printed above happens to be what the hardware does. It is not what the standard promises, and the optimiser is not obliged to agree with the hardware.

2.9 Narrowing is silent

```
int main()
{
  std::int64_t horizontal_velocity = 40000;   // fine on the old rocket

  std::int16_t converted = static_cast<std::int16_t>(horizontal_velocity);

  std::printf("velocity (64-bit)  = %ld\n", horizontal_velocity);
  std::printf("stored as 16-bit   = %d   <- sign flipped\n", converted);

  // Implicit narrowing compiles without a cast, and without a warning
  // unless you asked for one.
  int  big   = 300;
  char small = big;
  std::printf("int 300 -> char    = %d\n", small);
```

```
velocity (64-bit)  = 40000
stored as 16-bit   = -25536   <- sign flipped
int 300 -> char    = 44
```

A positive velocity becomes a negative one. This is, in miniature, the Ariane 5 Flight 501 failure of June 1996: a 64-bit floating-point horizontal-velocity value converted to a 16-bit signed integer, in inertial-reference software carried over unchanged from Ariane 4, where the value's range had been provably smaller. The conversion overflowed 37 seconds after ignition, the guidance computers shut down, and the vehicle was destroyed with its payload. The software was correct for the range it had been written against; nothing in the type system recorded what that range was.

2.10 The midpoint bug

```
int main()
{
  volatile int lo = INT_MAX - 3;
  volatile int hi = INT_MAX - 1;

  int mid_broken = (lo + hi) / 2;          // overflows
  int mid_safe   = lo + (hi - lo) / 2;     // does not

  std::printf("lo                 = %d\n", lo);
  std::printf("hi                 = %d\n", hi);
  std::printf("(lo + hi) / 2      = %d   <- outside [lo, hi]\n", mid_broken);
  std::printf("lo + (hi - lo) / 2 = %d   <- correct\n", mid_safe);
```

```
lo                 = 2147483644
hi                 = 2147483646
(lo + hi) / 2      = -3   <- outside [lo, hi]
lo + (hi - lo) / 2 = 2147483645   <- correct
```

The midpoint of two values inside the range lands outside it. This is the bug Jon Bentley described in *Programming Pearls* (1986) and Joshua Bloch rediscovered in the JDK in 2006, where `Arrays.binarySearch` had carried it for nine years. If a bug of this shape can live that long in one of the most scrutinised standard libraries in existence, written by people who knew about it, the problem is not insufficient care.

2.11 The semantic hazard: `int` does not know what it counts

Every hazard so far has been about range. This one is about meaning, and it is the one no sanitizer will ever catch, because the arithmetic is valid.

```
int price_in_cents(int item)
{
  static const int table[] = {1999, 450, 1234};
  return table[item];
}

int shipping_in_dollars()
{
  return 5;   // $5 flat rate
}

int main()
{
  int subtotal = price_in_cents(0) + price_in_cents(1) + price_in_cents(2);
  int total    = subtotal + shipping_in_dollars();   // <- units mismatch

  std::printf("subtotal (cents)   = %d\n", subtotal);
  std::printf("shipping (dollars) = %d\n", shipping_in_dollars());
```

```
  std::printf("total             = %d   <- charged 5 cents for shipping\n", total);
  std::printf("compiles clean, runs clean, bills the customer wrong.\n");
```

```
subtotal (cents)   = 3683
shipping (dollars) = 5
total              = 3688   <- charged 5 cents for shipping
compiles clean, runs clean, bills the customer wrong.
```

An `int` holding cents and an `int` holding dollars are the same type. Adding them is well-defined, warning-free, sanitizer-clean, and wrong. The Mars Climate Orbiter was lost in 1999 to precisely this class of error across a software interface — pound-force-seconds where newton-seconds were expected. No range check would have caught it, because no value was out of range. The information that would have caught it — what the number *means* — was never written down anywhere the compiler could see.

3 "Just use `double`"

The common response to the preceding section is to reach for floating point. `double` has a much wider range, it does not trap on division by zero, and it represents fractions. It is a reasonable instinct and it is the right tool for a genuine class of problems. But it is not a fix for the problems above; it substitutes a different family of silent wrongness, and this section is deliberately shorter than the last one only because the failures are more widely known — not because they are less serious.

3.1 Decimal fractions are not representable

```
0.1 + 0.2          = 0.30000000000000004441
0.3                = 0.29999999999999998890
0.1 + 0.2 == 0.3   ? false
```

Neither `0.1` nor `0.2` nor `0.3` exists in binary floating point. The literals are the nearest representable neighbours, and the sum of the first two is not the neighbour of the third.

3.2 The error accumulates

```
int main()
{
  double total = 0.0;
  for (int i = 0; i < 10000; ++i)
    total += 0.01;   // one cent, ten thousand times

  std::printf("10000 x $0.01      = %.20f\n", total);
  std::printf("expected           = %.20f\n", 100.0);
  std::printf("exactly $100.00    ? %s\n", (total == 100.0) ? "true" : "false");
  std::printf("error              = %.20f\n", total - 100.0);
```

```
10000 x $0.01      = 100.00000000001425348728
expected           = 100.00000000000000000000
exactly $100.00    ? false
error              = 0.00000000001425348728
```

Ten thousand transactions — a quiet morning for a payment system — and the ledger no longer balances to the cent. The error is small, which is precisely what makes it dangerous: it survives testing and shows up in reconciliation.

3.3 Addition is not associative

```
int main()
{
  double a = 1e16;
  double b = -1e16;
  double c = 1.0;

  std::printf("(a + b) + c        = %.1f\n", (a + b) + c);
  std::printf("a + (b + c)        = %.1f\n", a + (b + c));
  std::printf("equal              ? %s\n",
              (((a + b) + c) == (a + (b + c))) ? "true" : "false");
```

```
(a + b) + c        = 1.0
a + (b + c)        = 0.0
equal              ? false
```

The same three values, the same operator, two groupings, two answers. The practical consequence is that the result of a summation depends on the order the data was visited in. Parallelise a reduction across a different number of threads, or let the optimiser re-associate a loop, and the answer changes. Results stop being reproducible — not across platforms, but across *runs*.

3.4 Integers stop counting at 2^{53}

```
int main()
{
  double x = 9007199254740992.0;   // 2^53

  std::printf("2^53               = %.1f\n", x);
  std::printf("2^53 + 1           = %.1f   <- unchanged\n", x + 1.0);
  std::printf("2^53 + 1 == 2^53   ? %s\n", (x + 1.0 == x) ? "true" : "false");

  // Catastrophic cancellation: subtracting near-equal values destroys
  // every significant digit that mattered.
  double big = 1e16;
  std::printf("(1e16 + 1) - 1e16  = %.1f   (should be 1)\n", (big + 1.0) - big);
```

```
2^53               = 9007199254740992.0
2^53 + 1           = 9007199254740992.0   <- unchanged
2^53 + 1 == 2^53   ? true
(1e16 + 1) - 1e16  = 0.0   (should be 1)
```

Beyond 2^{53} a `double` cannot represent consecutive integers, so a counter silently stops incrementing — a real hazard for identifiers and timestamps in systems that pass numbers through JSON, where `double` is the only numeric type. The second line shows catastrophic cancellation: subtracting near-equal quantities annihilates every digit that mattered.

3.5 `NaN` breaks equality and ordering

```
0.0 / 0.0          = -nan
1.0 / 0.0          = inf   <- no error reported
nan == nan         ? false   <- breaks equality
nan < 1            ? false
nan > 1            ? false   <- neither; breaks sorting
nan + 1000         = -nan   <- contaminates the whole total
```

`double` does not trap, which is often described as an advantage. The cost is that an invalid operation produces a value that propagates silently through every subsequent computation, and that violates the strict-weak-ordering requirement `std::sort` relies on — sorting a range containing `NaN` is undefined behaviour, and can walk off the end of the buffer.

The honest summary is that floating point is excellent at what it was designed for: approximating continuous quantities across an enormous dynamic range. It is a poor fit for money, for counting, for identifiers, and for anything requiring reproducibility. Reaching for it to escape `int`'s problems means adopting a different set.

4 Why the usual defences fall short

Before proposing something new, it is worth being precise about why the existing tools do not close the gap.

Sanitizers are runtime instruments. UBSan genuinely detects signed overflow, and it is valuable. But it detects it only when the offending input actually occurs, in a build that carries the instrumentation, and it cannot detect what the optimiser already deleted. Running our very first example under UBSan:

```
$ g++ -std=c++23 -O2 -fsanitize=undefined h01_overflow_ub.cpp && ./a.out
h01_overflow_ub.cpp:24:14: runtime error: signed integer overflow:
    2147483647 + 1 cannot be represented in type 'int'
INT_MAX            = 2147483647
grows(INT_MAX)     = yes, always
INT_MAX + 1        = -2147483648
```

UBSan reports line 24 — the `printf` argument. It says nothing about `grows()`, where the comparison was folded away at translation time. There was no overflow left to instrument, because there was no arithmetic left. The diagnostic misses the defect that motivated the example.

`-ftrapv` and `-fwrapv` change the language. `-fwrapv` defines overflow as wrapping, which removes the undefined behaviour and the optimiser's licence with it. That is a real improvement, and it makes the wrong answer reliable rather than unpredictable: the midpoint bug still computes a negative midpoint. It is a fix for the UB, not for the arithmetic.

Assertions are manual, and unenforced. A precondition check is only present where somebody remembered to write it, expresses a range the type system does not know, and is frequently compiled out in release builds. It is documentation with a runtime cost, not a guarantee.

Static analysis is partial by construction. Range analysis across function boundaries and through data structures is undecidable in general. Analysers are useful and they are not complete; they cannot tell you that this `int` is cents and that one is dollars, because nothing in the program says so.

Safe-integer libraries solve one hazard. Wrapper types that check for overflow on every operation (`boost::safe_numerics`, `SafeInt`) genuinely fix hazards §2.1–§2.2. They do not address units, they do not address rounding policy, they do not give you a range narrower than the underlying type, and they detect at runtime what could have been rejected at compile time.

The common thread: each tool operates at a point where the necessary information has already been discarded. The range this value may occupy, the quantity it measures, the rounding rule that applies, and what should happen at the boundary are all decisions the programmer made and then failed to record in a form the compiler can act upon. The proposal in the next section is simply to record them.

5 `inside`: putting the range in the type

`inside` is a header-only C++23 library built around a single template:

```
template<grid G, policy_flag P>
struct inside;                          // include/beman/inside/core.hpp:83
```

The `grid` is a compile-time value carrying an interval (lower and upper inside) and a *notch* — the step size, held as an exact rational. Every value of the type is exactly Lower $+ k \cdot$ Notch for some integer k. The `policy` says what happens when a value would leave the range. Storage is selected automatically: the narrowest of `uint8`…`int64` that can number every point on the grid, or an exact fraction for a continuous grid, and grids too large for 64 bits get wider integers (§5.8). No flag overrides it: storage follows from the grid alone.

```
using namespace beman::inside;

using pct    = inside<{0, 100}>;                    // integers 0..100
using money  = inside<{{0, 1'000'000}, per<100>}, round_nearest>;  // cents
using angle  = inside<{0, 359}, wrap>;              // modular
using sample = inside<{{-1, 1}, per<16384>}, round_nearest>;       // Q1.14
```

Three properties follow, and between them they account for most of §2.

5.1 Arithmetic widens, so overflow is unreachable

The result type of an operation is computed at compile time from the operand grids. Adding two values in $[10, 255]$ yields a type whose range is $[20, 510]$ — wide enough that no representable input can overflow it. There is no overflow to be undefined, no check to perform, and nothing for the optimiser to exploit.

```
int main()
{
  using u8 = inside<{10, 255}>;
  constexpr u8 a{16};
  constexpr u8 b{220};

  // The sum of two [10,255] values lives in [20,510]. Checked at compile time.
  static_assert(a + b == 236);
  std::cout << "a + b              = " << (a + b) << "\n";

  // A difference that would underflow a native uint8 widens to signed instead.
  static_assert(a - b == -204);
  std::cout << "a - b              = " << (a - b) << "   (no wraparound)\n";

  // The midpoint computation that breaks binary search cannot break here:
  // lo + hi is computed in a grid wide enough to hold it.
  using idx = inside<{0, 2147483647}>;
  idx lo{2147483644};
  idx hi{2147483646};
  auto mid = (lo + hi) / just<2>;   // a plain value: 2 is not zero, and it fits
  std::cout << "(lo + hi) / 2      = " << mid << "   (still in range)\n";
```

```
a + b              = 236
a - b              = -204   (no wraparound)
(lo + hi) / 2      = 2147483645   (still in range)
```

Three things happened there. The sum was verified at compile time by `static_assert`, so a wrong answer would have been a build failure rather than a test failure. The difference of two *unsigned-ranged* values produced -204: because the result grid is $[-245, 245]$, the type widened to a signed representation rather than wrapping. And the midpoint computation from §2.10 — the one that produced -3 in `int` — stayed in range, because `lo + hi` was evaluated in a grid wide enough to hold it before the division.

5.2 Division by zero is a value, not a signal

Where the divisor's grid can include zero, division returns `std::expected<inside, errc>`. Where the grid provably excludes zero *and* the exact quotient provably fits its 64-bit rational — which the operand grids decide at compile time, and which holds for every integer grid — it returns a plain value with nothing to unwrap: both checks are elided because the types prove them unnecessary.

```
int main()
{
  using val = inside<{0, 100}>;

  val a = 7;
  val b = 3;

  // Exact rational result -- no silent truncation.
  auto exact = a / b;
  if (exact)
    std::cout << "7 / 3              = " << *exact << "   (exact, not 2)\n";

  // Opt in to C-style truncation explicitly, per call.
  auto trunc = div(a, b, snapped);
  if (trunc)
    std::cout << "div(7, 3, trunc)   = " << *trunc << "\n";

  // Division by zero: an error value, and the program keeps running.
  val zero = 0;
  auto oops = val(10) / zero;
  std::cout << "10 / 0             = "
            << (oops.has_value() ? "value" : errc_message(oops.error()))
            << "   (no SIGFPE)\n";
```

```
7 / 3              = 7/3   (exact, not 2)
div(7, 3, trunc)   = 2
10 / 0             = division by zero   (no SIGFPE)
```

Note the first line. The default result of division is *exact*: $7/3$ is represented as the rational $7/3$, not truncated to 2. Truncation is still available, but you ask for it by name, per call. The failure that terminated the process in §2.4 is now a value the caller can test. (Here the divisor's grid $[0, 100]$ contains zero, so the quotient arrives as `std::expected` and the program unwraps it; the next section divides by a zero-free grid and gets a plain value.)

5.3 Rounding is stated, not inherited

Native integer division gives $-7/2 = -3$, toward zero, always (§2.7). In `inside` the default is the exact answer, and a rounding is something you name:

```
  using val = inside<{-100, 100}>;
  using pos = inside<{1, 100}>;   // divisor grid excludes zero

  val n{-7};
  pos d{2};

  // The default is the exact quotient. The divisor cannot be zero and the
  // quotient provably fits, so it is a plain value: nothing to unwrap.
  std::cout << "n / d              = " << n / d << "   (exact)\n";

  // Or, if you ask for a rounding, the one you name -- per call.
  std::cout << "rounded_nearest    = " << div(n, d, rounded_nearest) << "\n";
  std::cout << "rounded_floor      = " << div(n, d, rounded_floor) << "\n";
  std::cout << "rounded_ceil       = " << div(n, d, rounded_ceil) << "\n";
  std::cout << "snapped (trunc)    = " << div(n, d, snapped) << "\n";
```

```
n / d              = -3.5   (exact)
rounded_nearest    = -4
rounded_floor      = -4
rounded_ceil       = -3
snapped (trunc)    = -3
```

The first line is the same mechanism as §5.2's 7/3: an exact quotient, printed as a terminating decimal where one exists and as a fraction where it does not. Because `pos` excludes zero and the quotient fits, it is a plain value. Each named mode is honoured, including for negative operands, where the distinction between truncation and flooring actually bites. The repository pins this with a property test asserting $(a/b)\cdot b + a \bmod b = a$ across whole grids for every mode, including negative divisors (`tests/beman/inside/div_round.test.cpp:147`).

5.4 The edge is a policy you declare

Out-of-range values are only reachable when assigning into a narrower type — never from arithmetic. What happens then is part of the declaration:

```
int main()
{
  // checked (the default): reports rather than storing a wrong value.
  try
  {
    checked_100 x = 200;
    (void)x;
  }
  catch (beman::inside::inside_error& e)
  {
    std::cout << "checked  200 ->    " << errc_message(e.Code) << "\n";
  }

  // clamp: saturate at the boundary.
  clamp_100 cl = 150;
  std::cout << "clamp    150 ->    " << cl << "\n";

  // wrap: modular arithmetic, on purpose this time.
  wrap_360 wr = 370;
  std::cout << "wrap     370 ->    " << wr << "\n";

  // try_make: failure is a value you test, not an exception.
  auto tm = digit::try_make(10);
  std::cout << "try_make  10 ->    " << (tm ? "has value" : errc_message(tm.error())) << "\n";
```

```
  // Per-operation override on an otherwise-checked value.
  checked_100 p{50};
  p.with_clamp() = 150;
  std::cout << "with_clamp() 150 -> " << p << "\n";
```

```
checked  200 ->    value does not fit its range
clamp    150 ->    100
wrap     370 ->    10
try_make  10 ->    value does not fit its range
with_clamp() 150 -> 100
```

The wrapping behaviour that `int` gives you unasked and undefined is available here as `wrap` — explicitly requested, well-defined, and producing $370 \to 10$ on a $[0, 359]$ grid rather than $\text{INT_MAX} + 1 \to \text{INT_MIN}$. The default is `checked`, which reports rather than storing a value it cannot represent — and every policy is checked unless it says `unsafe`: a rounding policy such as `round_nearest` rounds first and then still range-checks the result. Turning the check off is a word you have to write.

5.5 Negation reflects the type

The asymmetry of §2.3 disappears because negating a $[\text{Lower}, \text{Upper}]$ value yields a value of type $[-\text{Upper}, -\text{Lower}]$:

```
int main()
{
  using temp = inside<{-40, 125}>;   // an asymmetric range, on purpose

  constexpr temp coldest{-40};

  // -coldest has type inside<{-125, 40}> -- the interval is mirrored.
  static_assert(-coldest == 40);
  std::cout << "coldest            = " << coldest << "\n";
  std::cout << "-coldest           = " << (-coldest) << "   (representable by
    construction)\n";

  // The same holds at the extreme of the range: there is no value whose
  // negation falls outside the negated type.
  constexpr temp hottest{125};
  static_assert(-hottest == -125);
  std::cout << "-hottest           = " << (-hottest) << "\n";
```

```
coldest            = -40
-coldest           = 40   (representable by construction)
-hottest           = -125
```

There is no value whose negation falls outside the negated type, so `-INT_MIN` has no analogue. The same reasoning gives an `abs` that cannot return a negative number.

5.6 The scale is part of the type

The units failure of §2.11 is a failure to record what a number means. `inside` records the scale in the notch, so a value on a cent grid and a value on a dollar grid are different types, and combining them converts:

```
using money   = inside<{{0, 1'000'000}, per<100>}, round_nearest>;
using dollars = inside<{0, 1000}>;   // whole dollars only
```

```cpp
int main()
{
  money subtotal{0};
  subtotal += money{19.99};
  subtotal += money{4.50};
  subtotal += money{12.34};
  std::cout << "subtotal           = $" << subtotal << "\n";

  dollars shipping{5};   // "5" -- but 5 what? The type says: whole dollars.
  std::cout << "shipping           = $" << shipping << "\n";

  // Adding them scales the dollar value onto the cent grid automatically.
  auto total = subtotal + shipping;
  std::cout << "total              = $" << total << "   (not $36.88)\n";

  // A bare integer cannot enter the arithmetic at all: `subtotal + 5` is
  // ill-formed, because 5 carries no grid. See b06_compile_time.cpp.
```

```
subtotal           = $36.83
shipping           = $5
total              = $41.83   (not $36.88)
```

The `int` version of this program charged five cents for shipping. Here the dollar value is scaled onto the cent grid automatically, because the compiler knows what each grid's step size is.

Bare integers cannot enter the arithmetic at all — there is nothing to say what grid they belong to — and the diagnostic says so directly:

```
$ g++ -std=c++23 -I ../../include x01_bare_int_rejected.cpp
   16 |   auto bad = x + 1;      // error: no grid for the literal
include/beman/inside/arithmetic.hpp:518:31: error: static assertion failed:
   an inside cannot be combined with a raw scalar: give the scalar a grid --
   `1_ins`, `just<1>`, `one`, or `inside<{lo,hi}>{n}` for a runtime value with
   a known range
```

5.7 Bounded indices

```cpp
int main()
{
  std::array<int, 10> data{};
  for (std::size_t i = 0; i < data.size(); ++i)
    data[i] = static_cast<int>(i * i);

  using index = inside<{0, 9}>;

  // In range: usable directly.
  index i{7};
  std::cout << "data[7]            = " << data[i.to<std::size_t>().value()] << "\n";

  // Out of range: an error value, not a buffer overrun.
  auto bad = index::try_make(10);
  std::cout << "index 10           = " << (bad ? "has value" : errc_message(bad.error()))
            << "   (no out-of-bounds access possible)\n";

  // Iterating the valid slots exactly, with no off-by-one to get wrong.
  int sum = 0;
  for (auto k : inside_range<{0, 9}>{})
    sum += data[k.to<std::size_t>().value()];
  std::cout << "sum of all slots   = " << sum << "\n";
```

```
data[7]            = 49
index 10           = value does not fit its range   (no out-of-bounds access possible)
sum of all slots   = 285
```

The bounds check happens once, at construction, rather than at every access or not at all. An index that exists is in range by construction; one that would not be simply fails to be created.

5.8 No 64-bit ceiling

`int64` is the last stop for every integer in C++. Under C++23 an `inside` grid stops there too: its limits and notch are 64-bit fractions. Under C++26 with static reflection (GCC 16, `-freflection`) they have no size limit. The library interns large grid numbers in static storage (`std::define_static_array`), so equal grids stay the same type however they were spelled, and the raw storage becomes a fixed-width integer sized from the grid:

```
  // int64 stops at 9223372036854775807. A [0, 2^100] grid does not.
  using huge = inside<{0, 0x1p100}>;
  constexpr huge a = 1e30;                    // the double 1e30, an integer, held exactly

  // a * a lives in [0, 2^200]: the product grid is computed at compile time,
  // so the multiplication cannot overflow and returns a plain value.
  constexpr auto sq = a * a;
  std::cout << "a                  = " << a << "\n";
  std::cout << "a * a              = " << sq << "\n";
  std::cout << "sizeof(a), sizeof(a * a) = " << sizeof(a) << ", " << sizeof(sq) << " bytes\n";

  // A notch of exactly 10^-30, which no double and no int64 ratio can hold.
  using fine = inside<{{0, 1}, 1e-30_g}, round_nearest>;
  fine third = just<1>.per<3>;                    // rounded once, onto the 1e-30 grid
  std::cout << "1/3 on a 1e-30 grid = " << third << "\n";
```

```
$ g++-16 -std=c++26 -freflection -O2 -I ../../include b11_no_ceiling.cpp
a                  = 1000000000000000019884624838656
a * a              = 1000000000000000039769249677312000395398304974095154031886336
sizeof(a), sizeof(a * a) = 16, 32 bytes
1/3 on a 1e-30 grid = 0.333333333333333333333333333333
```

(The first line is not a typo: the literal `1e30` is a `double`, and that is its exact value. `inside` stores what it is given exactly; an exact 10^{30} is spelled `1e30_g`.) The product of two values in $[0, 2^{100}]$ lives in $[0, 2^{200}]$, so the same widening rule that made §2.1's overflow unreachable applies past any machine word; the multiplication returns a plain value and costs 32 bytes, not an allocation. The notch of 10^{-30} is exact — no `double` and no ratio of 64-bit integers can hold it. Compiled as C++23, the same file is rejected at compile time with a message naming the C++26 requirement, rather than silently truncating the grid.

6 A worked migration

An invoice calculation, in `int`. It has three defects, and the compiler mentions none of them.

```
int main()
{
  // A large but entirely plausible invoice: $2,684,354.56 in cents.
  int subtotal = 268435456;

  int tax      = subtotal * 8 / 100;   // 8% sales tax
  int discount = subtotal / 3;         // "a third off"
```

```
  int total    = subtotal + tax - discount;

  std::printf("subtotal (cents)   = %d\n", subtotal);
  std::printf("tax  (8%%)          = %d   <- overflowed\n", tax);
  std::printf("discount (1/3)     = %d   <- truncated\n", discount);
  std::printf("total              = %d\n", total);
```

```
subtotal (cents)   = 268435456
tax  (8%)          = -21474836   <- overflowed
discount (1/3)     = 89478485   <- truncated
total              = 157482135

expected tax       = 21474836.48
expected discount  = 89478485.33
```

The invoice is for $2,684,354.56 — large, but not implausible. Computing 8% of it as `subtotal * 8 / 100` overflows `int` at the multiplication, and the tax comes out *negative*. The discount silently loses its fractional part. The total is meaningless, and every intermediate looks like a perfectly ordinary integer.

The same calculation in `inside`:

```
// Dollars in 1-cent steps, up to $10M. The scale is part of the type.
using money = inside<{{0, 10'000'000}, per<100>}, round_nearest>;

int main()
{
  money subtotal{2'684'354.56};

  // subtotal * 0.08 widens; nothing can overflow. Assigning back into
  // `money` snaps to the nearest cent, because `money` says round_nearest.
  auto  tax_exact = subtotal * 0.08_ins;
  money tax       = tax_exact;

  // A third of the subtotal is not representable in cents. It stays exact
  // until it is stored, and then rounds by the stated rule.
  auto  disc_exact = subtotal * just<1>.per<3>;
  money discount   = disc_exact;

  money total = subtotal + tax - discount;

  std::cout << "subtotal           = $" << subtotal << "\n";
  std::cout << "tax  (8%)          = $" << tax      << "   (exact, no overflow)\n";
  std::cout << "discount (1/3)     = $" << discount << "   (rounded to nearest cent)\n";
  std::cout << "total              = $" << total    << "\n";

  // The intermediate really is exact: a grid in thirds of a cent stores
  // the same value with nothing rounded away.
  using third_cent = inside<{{0, 10'000'000}, per<300>}, round_nearest>;
  third_cent discount_exact = disc_exact;

  std::cout << "\ndiscount on a 1/300 grid       = $" << discount_exact
            << "   (nothing was lost)\n";
```

```
subtotal           = $2684354.56
tax  (8%)          = $214748.36   (exact, no overflow)
discount (1/3)     = $894784.85   (rounded to nearest cent)
total              = $2004318.07

discount on a 1/300 grid       = $67108864/75   (nothing was lost)
```

What changed, and what it cost:

- **The overflow is gone by construction.** `subtotal * 0.08_ins` widens to a grid that can hold the product. There is no check, because there is nothing to check.
- **The tax is exact to the cent.** `0.08` is parsed as an exact rational, not as the nearest `double`. The product is exact, and assigning it into `money` snaps to the nearest cent because `money` was declared `round_nearest`. Truncating instead would require saying so.
- **The third stays exact until it is stored.** The last line proves it: stored onto a grid of thirds-of-a-cent, the discount is 67108864/75 dollars exactly. The rounding in the line above it happened once, at the point of storage, by a stated rule — not silently at every intermediate step.
- **The friction is real but small.** Literals need a grid (`0.08_ins`), non-representable constants need `just<1>.per<3>` rather than `1.0/3.0`, and the declaration of `money` is a line of thought that the `int` version never had to have. That line is the point: it is where the range, the scale, and the rounding rule got written down.

7 Correctness

7.1 Errors caught before the program runs

The strongest claim `inside` makes is not that it detects errors but that it rejects them at compile time. Because grids are compile-time values and the arithmetic is `constexpr`, whole categories of defect become build failures.

```
// Range violations are knowable before the program exists.
static_assert(conversion_overflows<pct>(150));
static_assert(conversion_overflows<pct>(-1));
static_assert(!conversion_overflows<pct>(50));

// Saturating and wrapping casts are constant-evaluable.
static_assert(clamp_cast<pct>(150) == 100);
static_assert(wrap_cast<angle>(370) == 10);
static_assert(wrap_cast<angle>(-10) == 350);

// Arithmetic results are exact and checkable at compile time.
constexpr pct a{60};
constexpr pct b{55};
static_assert(a + b == 115);   // widened past 100, and known to be so
```

Every line above is a `static_assert`. If any were false the file would not build, and no test would need to run to discover it. An out-of-range constant is likewise rejected outright:

```
$ g++ -std=c++23 -I ../../include x02_range_rejected.cpp
error: 'constexpr void beman::inside::policy<W,E>::report(beman::inside::errc)'
       called in a constant expression
error: call to non-'constexpr' function 'void
    beman::inside::detail::raise(beman::inside::errc,
       const char*)'
```

The mechanism is worth understanding: the error path is deliberately non-`constexpr`, so reaching it during constant evaluation is automatically a compile error. The library's own constant-evaluation tests (`tests/beman/inside/constexpr.test.cpp`) are written on this basis: 158 `static_asserts` that run inside the compiler.

7.2 The error vocabulary

`inside` reports failure in one of several shapes, and which one you get is determined by how you spell the call. The design rule is that the shape matches the caller's role.

Shape	Spelling	When
Plain value	`a + b`	The type proves the operation cannot fail.
`expected<T,errc>`	`a / b`, `try_make`, `to<T>()`, `math::tan`, `math::pow`	The operation can fail at runtime; the caller tests the result and, if it cares, reads the cause. A math argument outside the function's domain for the whole input grid is a compile error instead.
Error code	`x.policy(ec) = v`	Hot paths, or bulk operations where the first error should stick.
Throwing	default (`checked`)	Programming errors that should not be handled locally.
Callback	`on_clamp`, `on_wrap`	The event is expected and carries data (a carry, an overshoot).

The codes themselves are a small closed enumeration (`detail/debug.hpp:52`), one code per condition — a value that does not fit its destination is always `overflow`, whichever path found it:

```
enum class errc { domain_error, division_by_zero, overflow,
                  rounding_error, not_finite, invalid_format };
```

There is no `<system_error>` dependency. Where exceptions are unavailable, `set_error_handler` replaces the failure path globally, which is what allows the library to run freestanding on bare metal.

The callback shape is more useful than it first appears, because some "errors" are the actual semantics of the domain. A clock is the canonical case: seconds overflowing 59 is not a fault, it is a carry.

```
void add_seconds(auto n)
{
  seconds.on_wrap([&](auto, auto carry) { add_minutes(carry); }) += n;
}
```

That is the whole of the carry logic from `examples/clock.cpp` — the wrap event delivers the overshoot to the next field up, and the cascade to minutes and hours falls out of the same pattern.

7.3 Exactness

Because the notch is an exact rational rather than a binary fraction, grids like "cents" and "thirds" are represented without approximation. The money example from §3 run in `inside`:

```
  // 0..$1,000,000 in 1-cent steps. 0.01 is not exact in binary, so the
  // notch is given as an exact rational -- not as 0.01.
  using money = inside<{{0, 1'000'000}, per<100>}, round_nearest>;
  static_assert(sizeof(money) == 4);   // still four bytes

  money total{0};
  for (int i = 0; i < 10000; ++i)
    total += money{0.01};

  std::cout << "10000 x $0.01      = $" << total << "\n";
  std::cout << "exactly $100.00    ? "
            << (total == money{100.00} ? "true" : "false") << "\n";
```

```
  // Read the value back out exactly, as an integer pair.
  std::cout << "as a fraction      = " << total.numerator()
            << "/" << total.denominator() << "\n";
```

```
10000 x $0.01      = $100.00
exactly $100.00    ? true
as a fraction      = 100/1
```

Ten thousand additions of one cent produce exactly one hundred dollars, in four bytes of storage — the same computation that drifted by 1.4×10^{-11} in `double`. The `static_assert(sizeof(money) == 4)` in that listing is doing real work: exactness here does not require arbitrary-precision arithmetic, because the grid is known at compile time and the value is stored as an integer count of notches.

There is no floating-point storage at all. A `double` enters an `inside` through one exact rounding onto its grid and leaves it through one correctly rounded conversion; in between, the value is an integer.

7.4 Reproducibility

For integer and rational storage, `+ - * /` are reproducible unconditionally — the repository's determinism document states it plainly: "There is no floating point on these paths, so there is nothing for the platform to round differently." Conversion from `double` decomposes the value from its IEEE-754 bits rather than computing with it, so it needs no FPU and no `<cmath>`.

For transcendentals, `beman::inside::math` goes one step further: every function returns the *correctly rounded* point of its output grid, under the output's rounding mode. A correctly rounded result is a function of the input value and the output grid alone, so it cannot depend on how it was computed. The repository's determinism document lists the conditions for the transcendentals as "**Always** — every result is the correctly rounded grid point, the same at compile time, at runtime, with or without an FPU."
The engine gets there by computing each result together with a proved error bound, at the precision the output grid needs, and escalating (Ziv's strategy) whenever the bound does not place the result in a single slot. Cheap tiers — compile-time tables for small input grids, the library's own `double` and double-`double` kernels with bounds proved at compile time — answer most calls; an integer path answers the rest, and is the only path at compile time and in builds without an FPU. All of them land on the same grid point. Measured against a long-double reference, every function's maximum error is 0.5000 notches (`docs/accuracy.md`): the correctly rounded answer, every time.

Two consequences are worth stating. First, there is no "engine choice" left in the reproducibility contract: the earlier library had three engines that were each reproducible but not value-identical to one another, and that caveat is gone. Second, the guarantee rests on IEEE arithmetic as written, so `-ffast-math` builds are refused with an `#error` rather than allowed to break it silently. The price of correct rounding is paid in time, and §8.6 measures it.

8 Performance

An abstraction that costs nothing is suspicious. This one does not have a single cost at all, and the most important thing to say about its performance is that **the cost is not a number, it is a range with a negative end**. Depending on what the code is doing, using `inside` instead of `int` can make the program faster, leave it unchanged instruction-for-instruction, or make it substantially slower. All three regimes are measured in this section:

Regime	When	Measured
Negative cost	A chain of operations whose intermediate ranges the compiler can discharge, replacing checks the equivalent hand-written code must perform at runtime; comparisons the grid lets run on raw storage.	22% faster than native checked (§8.4); comparison against a scalar at 135% of native
Zero cost	Integer-aligned operands on the same grid, where all the range reasoning happens in the type system.	Byte-identical disassembly (§8.1); `inside<unsafe>` at 99.7% (§8.3); add, construct, `std::accumulate`, sort, transform at 95–101%
Additional cost	Rational storage (continuous grids), element-wise checking, and transcendentals correctly rounded at full `double` resolution.	rational add 2–5× `double`; checked loop at 20%; math at full `double` resolution 4–9× `std`

Which regime you land in is mostly determined by how the code is written rather than by the library: staying in inside-space lets the grid arithmetic discharge obligations, while round-tripping through raw `int` at every step forces a check at each boundary and forfeits the advantage. A single headline number — in either direction — would be a misrepresentation, so this section reports all three with the programs that produced them.

8.1 The fast path is byte-identical to native

The headline claim is that when both operands are integer-aligned on the same grid, the operation lowers to a plain machine instruction. That claim is testable, so here is the test:

```
using val = inside<{0, 1000}, unsafe>;

// The inside version.
[[gnu::noinline]] val inside_add(val a, val b)
{
  return val{a.raw() + b.raw()};
}

// The native version, for comparison.
[[gnu::noinline]] int native_add(int a, int b)
{
  return a + b;
}
```

```
$ g++ -std=c++23 -O2 -I ../../include -c p01_codegen.cpp && objdump -d p01.o

0000000000000000 <inside_add(inside<...>, inside<...>)>:
   0:   endbr64
   4:   lea    (%rdi,%rsi,1),%eax
   7:   ret

0000000000000010 <native_add(int, int)>:
  10:   endbr64
  14:   lea    (%rdi,%rsi,1),%eax
  17:   ret
```

The two functions are byte-identical. All of the range reasoning happened in the type system and left no trace in the object code. Loops over these types still vectorise: both the `inside` and the

native loop emit packed SIMD adds, the `inside` version on 16-bit lanes (its storage for $[0, 1000]$ is narrower than `int`) and the native version on 32-bit lanes.

This is not an accident that could regress unnoticed. The repository's test suite enforces it: `tests/beman/inside/check_codegen.sh` compiles seven named kernels at `-O2`, disassembles them, and **fails if any of them contains a `call` instruction** other than the cold error report, and if the loop does not vectorise. It runs in CI on every compiler and standard library in the matrix. A cachegrind workload (`tests/beman/inside/perf_workload.cpp` with `check_perf.py`) guards instruction counts locally.

8.2 What it costs, and against what

The tables below are extracted from the repository's generated benchmark report, `docs/performance.md` (GCC 15, `-O3`, one run pinned to one core). **Read the *rel.* column as a fraction of the first row's throughput**: 100% is parity with that row, and smaller is slower. Absolute nanosecond figures are host-specific; the report itself warns that CPU frequency scaling and turbo were active during the run, so the ratios — not the times — are the signal, and sub-nanosecond rows move by 15–20% from run to run. The instruction counts do not.

The choice of baseline matters more than any individual number, and it is easy to read these tables unfairly in `inside`'s favour or against it. **The 100% row is always plain, unchecked native code**: a bare cast and an add, with no range enforcement of any kind. It is the fastest thing in the table because it is not doing the job. It is the code from §2 — the code that overflows silently, that lets 300 become 44, that computes a negative midpoint. Comparing `inside` against it measures the cost of correctness that the baseline never pays.

The benchmark therefore also carries two *hand-written native* baselines that do enforce a range, and these are the honest comparison (`tests/beman/inside/bench.test.cpp:59`):

- `native clamped` — the same arithmetic wrapped in `std::clamp(v, 0, 200)`.
- `checked (exceptions)` — a minimal hand-rolled bounded integer struct that range-checks in its constructor and in `operator+`, throwing `std::out_of_range` on violation.

Neither of those is `inside`; both are ordinary C++ a competent programmer would write to get the guarantee `inside` provides. In the tables below the `inside` rows are the ones named `inside<...>`, `beman::inside::`, or `inside`; everything else is a native reference point.

8.3 The like-for-like comparison

The generated report does not pair `inside<checked>` against an equivalently-checked native implementation for the same operation, so the question "what does `inside` cost against code that does the same job?" cannot be answered from it directly. `p02_fair_bench.cpp` in the companion directory answers it by measuring all five contenders on the same data in the same program: a summed pass over 4096 values, repeated 20,000 times, with inputs chosen so that no contender ever actually throws.

Implementation	ns/op	vs. unchecked	enforces?	vs. safe native
`native unchecked`	0.436	100.0%	no	—
`native clamped`	0.457	95.4%	yes	100.0%
`native checked`	0.511	85.3%	yes	89.4%
`inside<checked>`	0.525	83.1%	yes	**87.1%**
`inside<unsafe>`	0.438	99.7%	no	—

Mean of three runs pinned to one core; GCC 15.2.0, `-O2`, x86-64. The final column is relative to `native clamped`, the fastest native implementation that actually enforces the range.

Read against the correct baseline the result is very different from what the unchecked column suggests:

- `inside<checked>` runs at **87.1%** of the fastest safe native code, and at **97.3%** of the hand-rolled checked struct — within noise of the code you would have written by hand to get the same guarantee.
- `inside<unsafe>` runs at **99.7%** of unchecked native, which is the same conclusion the disassembly in §8.1 reached by a different route.
- The cost of the guarantee itself, in plain C++ with no library involved, is 5–15%. That is most of what the `inside<checked>` row is paying for. The library's own overhead on top of it is about 3%.

This is a favourable measurement pattern and it should be labelled as one: a tight loop over contiguous data, where every contender vectorises and the branch predictor sees the same outcome every iteration. It is not evidence that `inside` is free in all circumstances — the tables that follow show cases where it is decidedly not. It is evidence for the narrower claim that *when you compare against code doing the same work, the gap is small*, and that most of the dramatic slowdowns reported against "native" are measuring the absence of a safety check rather than the presence of a library.

8.4 Where are the checks?

The table above still shows `inside<checked>` marginally behind the hand-rolled native struct, which is a fair thing to be suspicious of: `inside` holds strictly more compile-time information, so it should be able to prove some checks unnecessary and delete them. It does. The reason it gains nothing in that particular benchmark is that the benchmark gives it nothing to prove — every iteration converts a raw `int` into an inside and immediately narrows the result back, which is the one shape where a check is genuinely unavoidable.

Disassembling each phase separately shows exactly where a check is emitted and where it is not:

Operation	Emitted	Check?
`bchecked(int v)`	`cmp $0xc8,%edi; ja ...`	yes
`auto c = a + b` (kept wide)	`movzbl; movzbl; lea; ret`	**none**
`bchecked c = a + b`	`add; cmp $0xc8,%rax; jg ...`	yes
`checked_u8 c = a + b` (native)	`add; cmp $0xc8,%eax; jg ...`	yes
`bchecked b = a` (same grid)	`mov; ret`	**none**
`wide w = a` (widening)	`movzbl; ret`	**none**

A check appears in exactly two places: entering `inside` from a raw integer whose range nothing knows, and narrowing a wide result into a smaller grid. Everywhere the grid arithmetic can discharge the obligation — same-grid copies, widening assignments, and every intermediate in an expression — the check is gone, and `inside`'s narrowing check is instruction-for-instruction what the native struct emits.

The consequence shows up as soon as there is a *chain* of operations rather than a single one. A native checked type must re-validate after every step, because each intermediate has to fit back into the checked type. `inside` lets the intermediates widen into grids computed at compile time and checks only the final store. Summing four values in $[0, 50]$ into a total declared as $[0, 200]$ (`p03_check_elision.cpp`):

Implementation	ns/op	vs. unchecked	runtime checks
`native unchecked`	0.871	100.0%	0
`native checked`	1.065	81.8%	3
`inside<checked>`	0.873	**99.8%**	**0**

`inside<checked>` is **22% faster than the native checked type** here, and within 0.3% of native code that performs no checking at all — while providing the guarantee that the unchecked version does not. Zero checks, not one: four values in $[0, 50]$ sum to $[0, 200]$, which is exactly the declared range of the result, so the final store is proved safe too. The whole function is:

```
endbr64
add    %esi,%edi
add    %edx,%edi
lea    (%rdi,%rcx,1),%eax
ret
```

That is the same two adds and an `lea` that GCC emits for the unchecked native chain. The native checked version, by contrast, emits three `cmp`/`jg` pairs, a cold path per throw site, and enough register pressure to spill.

The proof is real rather than skipped: narrow the result type by one, to $[0, 199]$, and the sum no longer provably fits — the store check (`cmp $0xc7,%al; ja`) reappears in the disassembly immediately. This is the answer to "why use a library instead of writing the checks by hand": the hand-written checks are the ones that cannot be removed, because nothing in `int` records what has already been established.

8.5 The generated tables

construct (u8 [0,200])

rel.	ns/op	ins/op	enforces?	implementation
100.0%	0.30	7.00	no	`native uint8`
72.9%	0.41	13.00	yes	`native clamped`
71.4%	0.42	9.00	yes	`native checked (exceptions)`
101.1%	0.30	7.00	no	`inside<unsafe>`

add (u8 [0,200])

rel.	ns/op	ins/op	enforces?	implementation
100.0%	0.41	11.00	no	`native uint8`
50.7%	0.81	15.00	yes	`native clamped`
79.7%	0.51	17.00	yes	`native checked (exceptions)`
99.2%	0.41	10.00	no	`inside<unsafe>`

Construction and addition with the check switched off are native: the same instruction count, give or take one. (The first edition of this paper measured 34% and 17% here; the difference is the library, not the benchmark.)

increment (u32 [0,200000])

rel.	ns/op	ins/op	implementation
100.0%	0.21	4.00	`native ++`
12.0%	1.71	6.00	`inside ++`

Increment is the outlier, and it is worth saying why, because it is not the library's arithmetic. Both rows run with the check off, and as plain functions on registers the native and `inside` increments compile to identical code (`lea; xor; cmp; cmovae`). In the benchmark the counter lives in a static variable across calls, and there GCC chooses a predicted branch for the native reset and a conditional move for the `inside` one; the conditional move keeps the comparison on the loop-carried dependency through memory, and the loop runs eight times slower. It is a code-generation choice that a microbenchmark exposes, reported here because it is in the table.

accumulate 1000 (u32 [0,200000], per element)

rel.	ns/op	ins/op	implementation
100.0%	0.05	1.03	`native loop`
99.8%	0.05	1.03	`inside<unsafe> loop`
19.9%	0.26	9.01	`inside<checked> loop`
47.0%	0.11	2.47	`beman::inside::sum<checked> (bulk check)`
99.1%	0.05	1.03	`std::accumulate native`
99.3%	0.05	1.03	`std::accumulate inside`

The accumulate table carries the single most actionable result in the report. An `inside<checked>` element-wise loop runs at 19.9% of native because it range-checks every element, which also stops the loop from vectorising. Using `beman::inside::sum`, which validates the total instead of each addend, moves it to 47.0% — a 2.4× improvement from choosing the right API for the job. Without the check, a hand loop and `std::accumulate` over `inside` both vectorise exactly like native code.

comparison ==

rel.	ns/op	ins/op	implementation
100.0%	0.55	12.00	`native int16`
103.8%	0.53	12.00	`inside same grid`
134.7%	0.41	9.00	`inside == int scalar (value raw)`
135.1%	0.41	9.00	`inside Q8.8 == int scalar (index raw)`

Comparison against a scalar beats native, with *fewer* instructions (9 against 12), because the grid tells the compiler the comparison can be done on raw storage without a conversion.

rational add (continuous grid)

rel.	ns/op	ins/op	implementation
100.0%	4.83	97.20	`native fraction`
371.8%	1.30	12.00	`std double (reference)`
180.6%	2.68	70.93	`inside (continuous)`

Rational arithmetic, checked for overflow, beats a hand-rolled reduced `int64` fraction and costs 2–5× a `double` (10 cycles; the `double` row measures 2 to 4 across runs) — the price of the exactness demonstrated in §7, not an inefficiency. Only a continuous grid pays it: any notched grid, thirds included (`per<3>`), stores an exact integer index and pays none of this.

sort 10k (per element)

rel.	ns/op	ins/op	implementation
100.0%	15.07	104.12	`native int16`
95.2%	15.82	99.27	`inside`

transform v+1 10k (per element)

rel.	ns/op	ins/op	implementation
100.0%	0.01	0.32	`native uint8`
102.0%	0.01	0.32	`inside u255 (8-bit storage)`

8.6 What correct rounding costs

`beman::inside::math` returns the correctly rounded point of its *output* grid, and its cost follows that grid: a coarse output needs few bits, a fine one needs many. The generated report measures

the functions onto the output grid deduced from a 2^{-14} input, and there they often keep pace with `<cmath>`:

Function (deduced grid)	`std` ns	`inside` ns	rel.
`sin`	3.74	4.71	79.4%
`hypot`	2.97	3.13	94.8%
`atan2`	7.77	7.51	103.5%
`log`	2.63	5.40	48.7%
`sqrt`	1.23	2.20	55.6%
`exp`	2.89	6.95	41.5%

That comparison flatters the library, and should be read with the grid in mind: `std::sin` computes to a `double`'s 53 bits, while the `inside` rows compute to their deduced output grids — 15 bits for `sin` ($[-1, 1]$ in steps of 2^{-14}), 12 for `log`, 17 for `sqrt`, 26 for `exp`. The fair question is what correct rounding costs at the resolution of a `double` itself. `p04_math_cost.cpp` measures both, onto output grids with close to 2^{53} slots:

Function	`std` ns	deduced grid ns	`double`-fine ns	fine vs. `std`
`sin` (2^{-52})	3.41	4.84	14.6	4.3×
`exp` (2^{-40} on $[0, 4096]$)	2.67	7.12	12.5	4.7×
`log` (2^{-48})	2.34	5.60	17.4	7.4×
`sqrt` (2^{-51})	1.23	2.41	10.6	8.6×

Mean of three runs pinned to one core; GCC 15.2.0, `-O3 -mfma`, x86-64.

At full `double` resolution, correct rounding costs **4 to 9**× `std`. That buys results that are correct to the last bit and identical on every platform, compiler, optimisation level and FPU or none — properties `<cmath>` does not offer. It is a real trade, and it should be made deliberately: ask for the output grid you need, not the finest one available.

8.7 Reading these numbers honestly

Five conclusions, stated plainly:

1. **There is no single cost.** The measurements in this section span negative cost (22% faster than checked native, §8.4), zero cost (byte-identical codegen, §8.1) and real cost (2–5× for rationals, up to 9× for correctly rounded math at full resolution). Any summary of the form "`inside` costs N%" is wrong in at least one direction; the honest question is always *which regime does my code sit in.*

2. **Compare against code that does the same job.** The single largest source of misleading numbers here is the unchecked native baseline. It is the fastest row in every table because it enforces nothing — it is the code whose failures fill §2. Measured against native code that actually enforces the range, `inside<checked>` runs at 87.1% (§8.3), and on a chain of operations it is 22% *faster* than the checked native type (§8.4).

3. **The fast paths are genuinely free.** Aligned integer arithmetic, scalar comparison, sorting and generic algorithms compile to the same instructions as `int`, verified by disassembly and guarded by the test suite. `inside<unsafe>` measured at 99.7% of unchecked native in §8.3, by a different method, which is the same answer.

4. **Some paths genuinely cost, and no baseline choice rescues them.** Rational arithmetic is 2–5× `double`; a loop that checks every element runs at 20% of native; correctly rounded transcendentals at full `double` resolution are 4–9× `std`. These buy exactness, a range guarantee and bit-exact reproducibility respectively, and if you do not need those properties you should not pay for them.

5. **API choice dominates policy choice.** Moving from a checked element-wise loop to `beman::inside::sum` gained 2.5×; choosing a coarser output grid for a transcendental can gain 10×. Reach for the bulk operations and the right grid before reaching for the `unsafe` policy.

The imported tables are a single run on one machine with frequency scaling enabled; §8.3, §8.4 and §8.6 are means of three runs of programs included in the companion directory. All of them establish orders of magnitude, not precise ratios. Anyone making a decision on the margin should re-run `perf_report` and the `p0*` programs on their own hardware.

9 When not to use this

Intellectual honesty requires the other side of the ledger.

- **It is not yet ready for production.** The README says so (the Beman "under development" maturity level); the public API may change between versions. That is a real adoption risk for long-lived code.
- **It requires C++23**, which rules out a good deal of existing work: GCC 14+, Clang 19+ (Clang 18 with libc++). MSVC is not supported. Grids past 64 bits need C++26 static reflection, today GCC 16 with `-freflection`.
- **Compile times increase.** The arithmetic is a compile-time computation over grids; that work has to happen somewhere, and it happens in your build.
- **Error messages are template errors.** The library invests in this — the `x01` diagnostic quoted in §5 is a hand-written message naming the fix — but a mistake off the guided path still produces the kind of output C++ templates produce.
- **Widening surprises `auto` users.** The type of `a + b` is not the type of `a`. This is the mechanism that makes overflow unreachable, and it means `auto` variables in a chain of arithmetic acquire progressively wider types. That is correct, and it is not what most C++ programmers expect.
- **Correct rounding is not free.** The transcendentals return the correctly rounded grid point, and at the resolution of a `double` that costs 4–9× `<cmath>` (§8.6). Code that needs `double` throughput and tolerates `double` error should call `<cmath>`.
- **Not everything needs it.** A loop counter over a small container is not where numeric bugs come from. The cases that justify the machinery are the ones where a wrong answer is expensive: money, physical units, fixed-point signal processing, embedded registers, control loops, anything crossing a system boundary.

10 Conclusion

The defects catalogued in §2 are not exotic. They are the arithmetic behaviour of the type that C++ programmers reach for by default, several million times a day. They are not caused by carelessness — the midpoint bug survived nine years in the JDK, and Ariane 5 was destroyed by code that had been correct on the previous vehicle. They persist because the information required to catch them is absent from the program at the point where it would be needed.

`int` does not know its range, so it cannot check it. It does not know what it counts, so it cannot object to being added to something else. It has no rounding policy, so it truncates. It has no representation for failure, so division by zero kills the process. And because its overflow is undefined rather than merely wrong, the compiler is entitled to reason from the assumption

that the failure cannot occur — which is how a program that should print `3` becomes a `main` containing no instructions.

Writing those four facts into the type is not a large idea, and it does not require a new language. `inside` demonstrates that a range, a step size and an edge policy are enough to make overflow unreachable rather than merely detected, to make division by zero a value, to make rounding a decision, and to make a unit mismatch a compile error — while emitting, on its fast paths, byte-for-byte the same instructions as the `int` it replaces.

Dijkstra's closing argument was that the quality of a programmer is a decreasing function of the density of `goto` statements in their programs. That is not the claim here. The claim is narrower and easier to check: for any numeric quantity in your program, you already know its range, its units, and what should happen at its limits. Writing that down costs a line. Not writing it down costs whatever the bug costs.

A Building the examples

Every program cited in this paper is in `docs/int-considered-harmful/` in the repository.

```
# the int and double hazard programs need nothing but a compiler
g++ -std=c++23 -O2 h01_overflow_ub.cpp -o h01
g++ -std=c++23 -O0 h01_overflow_ub.cpp -o h01_O0    # compare the two

# the inside programs need the library on the include path
g++ -std=c++23 -O2 -I ../../include b01_no_overflow.cpp -o b01

# b11 needs C++26 static reflection
g++-16 -std=c++26 -freflection -O2 -I ../../include b11_no_ceiling.cpp -o b11

# the committed single header holds io.hpp and cmath.hpp too: to build
# against it, include only <beman/inside/inside.hpp> and use
#   -I ../../single_include

# the two x*.cpp files are expected to FAIL to compile, by design
g++ -std=c++23 -I ../../include x01_bare_int_rejected.cpp    # error, as intended
```

The library itself builds with CMake 3.30+, through its presets:

```
cmake --preset gcc-release && cmake --build --preset gcc-release
ctest --preset gcc-release
ctest --preset gcc-release -L example    # run the 36 example programs
```

B Program index

File	Demonstrates
`h01_overflow_ub.cpp`	Overflow check folded away, even at `-O0`
`h02_overflow_loop.cpp`	`main` compiled to no instructions
`h03_intmin.cpp`	`-INT_MIN` and `abs(INT_MIN)` negative
`h04_divzero.cpp`	`SIGFPE` on `x/0` and `INT_MIN/-1`
`h05_signed_unsigned.cpp`	Loop over an empty container
`h06_promotion.cpp`	Promotion, `char` signedness, `int` width
`h07_truncdiv.cpp`	Truncation toward zero, negative remainder
`h08_shift.cpp`	Shift into the sign bit
`h09_units.cpp`	Cents added to dollars
`h10_binsearch.cpp`	Midpoint outside the range
`h11_narrowing.cpp`	Silent narrowing (Ariane 5)
`d01_decimal.cpp`	`0.1 + 0.2 != 0.3`
`d02_money_drift.cpp`	Accumulated drift on a running total
`d03_assoc.cpp`	Addition is not associative
`d04_precision.cpp`	2^{53}; catastrophic cancellation
`d05_nan.cpp`	`NaN` breaks equality and ordering
`b01_no_overflow.cpp`	Widening arithmetic; the midpoint, fixed
`b02_divzero.cpp`	Division as `expected`; exact by default
`b03_policies.cpp`	`checked` / `clamp` / `wrap` / `try_make`
`b04_money.cpp`	Exact cents in four bytes
`b05_units.cpp`	Scale in the type
`b06_compile_time.cpp`	Range errors as build failures
`b07_rounding.cpp`	Every rounding mode honoured
`b08_index.cpp`	Bounded indices
`b10_negation.cpp`	Negation reflects the grid
`b11_no_ceiling.cpp`	Grids past 64 bits (C++26)
`m_before.cpp` / `m_after.cpp`	The worked migration
`x01_bare_int_rejected.cpp`	Bare literal rejected (expected failure)
`x02_range_rejected.cpp`	Out-of-range constant rejected (expected failure)
`p01_codegen.cpp`	Fast path vs. native disassembly
`p02_fair_bench.cpp`	Like-for-like benchmark vs. checked native
`p03_check_elision.cpp`	Chain of ops: inside elides checks native cannot
`p04_math_cost.cpp`	What correct rounding costs, coarse and fine

C References

1. E. W. Dijkstra, "Go To Statement Considered Harmful," *Communications of the ACM*, 11(3):147–148, March 1968.

2. J. Bentley, *Programming Pearls*, Addison-Wesley, 1986 — the binary search midpoint.

3. J. Bloch, "Extra, Extra — Read All About It: Nearly All Binary Searches and Mergesorts are Broken," Google Research Blog, June 2006.

4. J. L. Lions et al., *ARIANE 5 Flight 501 Failure: Report by the Inquiry Board*, ESA/CNES, July 1996.

5. NASA, *Mars Climate Orbiter Mishap Investigation Board Phase I Report*, November 1999.

6. D. Goldberg, "What Every Computer Scientist Should Know About Floating-Point Arithmetic," *ACM Computing Surveys*, 23(1):5–48, 1991.

7. ISO/IEC 14882, *Programming languages — C++*, §7.1 [expr.pre] (undefined behaviour on

overflow), §7.4 [conv.integral].

8. A. Ziv, "Fast evaluation of elementary mathematical functions with correctly rounded last bit," *ACM Transactions on Mathematical Software*, 17(3):410–423, 1991.

9. The `inside` repository, https://github.com/PeterNeiss/inside: `README.md`, `docs/tutorial.md`, `docs/policies.md`, `docs/arithmetic.md`, `docs/storage.md`, `docs/math.md`, `docs/determinism.md`, `docs/accuracy.md`, `docs/performance.md`.